

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

EXPLORATORY OF DATA ANALYSIS

LAB ASSESSMENT 8

NAME : ARJUN MISHRA

REG NO : 23BDS0334

SLOT : L31+L32

COURSE CODE : BCSE331P

ARJUN MISHRA – 23BDS0334

Assessment 8: K-Means Clustering

1. Load the dataset shared through e-mail.

2. View the dataset

3. Check missing values

4. Check for the dependent variable distribution. (table)

5. Select only numerical features for K-Means clustering (Others remove it).

6. Standardize the data. (scale)

7. Apply K-Means Clustering (kmeans) for center as 2.

8. Display K-Means results

9. Display cluster centers

10. Calculate WCSS for K = 1 to 10

11. Display the elbow plot.

12. Apply K-means clustering based on the elbow recommendation.

Hierarchical Clustering

1. Load the dataset shared through e-mail.

2. View the dataset

3. Check missing values

4. Select only numerical attributes

5. Standardize the data (scale)

6. Calculate Euclidean distance (dist)

7. Perform Hierarchical Clustering using Ward's method (hclust)

8. Plot the dendrogram. (plot)

9. Divide the data into 3 clusters and Display cluster assignments(cutree)

10. Display number of samples in each cluster. (table)

Part(A) K-Means Clustering

AIM:

CODE:

assessment_8.R | data | num_data

Source on Save

```r
#23BDS0334
#ArjunMishra
# ============================================
# ASSESSMENT 8: K-MEANS CLUSTERING
# ============================================

# 1. Load the dataset
data <- read.csv("assessment.csv")

# 2. View the dataset
View(data)
head(data)
str(data)

# 3. Check missing values
colSums(is.na(data))

# 4. Check dependent variable distribution
# Replace 'Target' with your dependent variable name
table(data$Target)

# If you don't know the dependent variable, view column names
names(data)

# ============================================
# 5. Select only numerical features
# ============================================

num_data <- data[, sapply(data, is.numeric)]

# View numerical data
View(num_data)

# ============================================
# 6. Standardize the data
# ============================================

scaled_data <- scale(num_data)

# View standardized data
head(scaled_data)
```

```r
# ==========================================
# 7. Apply K-Means with K = 2
# ==========================================

set.seed(123)

kmeans_2 <- kmeans(scaled_data, centers = 2, nstart = 25)

# ==========================================
# 8. Display K-Means results
# ==========================================

kmeans_2

# Cluster assignment for every observation
kmeans_2$cluster

# ==========================================
# 9. Display cluster centers
# ==========================================

kmeans_2$centers

# ==========================================
# 10. Calculate WCSS for K = 1 to 10
# ==========================================

wcss <- numeric(10)

for (k in 1:10) {

  set.seed(123)

  km <- kmeans(scaled_data,
               centers = k,
               nstart = 25)

  wcss[k] <- km$tot.withinss
}

# Display WCSS values
wcss

# ==========================================
# 11. Display Elbow Plot
```

```r
# ============================================
# 11. Display Elbow Plot
# ============================================

plot(1:10,
     wcss,
     type = "b",
     pch = 19,
     xlab = "Number of Clusters (K)",
     ylab = "WCSS",
     main = "Elbow Method for K-Means")

# ============================================
# 12. Apply K-Means based on elbow
# ============================================

# Change this value according to the elbow plot
optimal_k <- 3

set.seed(123)

final_kmeans <- kmeans(scaled_data,
                       centers = optimal_k,
                       nstart = 25)

# Display final result
final_kmeans

# Display final cluster assignments
final_kmeans$cluster

# Display final cluster centers
final_kmeans$centers

# Number of observations in each cluster
table(final_kmeans$cluster)
```

OUTPUT:

Filter

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	Cobble
1	1	0	786	190	10	0.1	0	3	0	0	0
2	2	0	794	182	10	0.0	0	3	0	0	0
3	3	0	798	178	20	0.0	0	1	0	0	0
4	4	0	800	176	70	0.3	0	1	0	1	0
5	5	0	804	172	40	0.3	0	1	0	1	0
6	6	0	808	168	60	0.3	0	1	0	1	0
7	7	0	814	162	40	0.3	0	1	0	1	0
8	8	0	817	159	0	0.9	1	1	1	1	0
9	9	0	821	155	50	0.5	0	1	0	1	0
10	10	0	826	150	20	0.1	0	1	0	0	0
11	11	0	829	147	0	0.7	0	1	1	1	0
12	12	0	834	142	10	0.9	1	1	0	1	0
13	13	0	839	137	0	0.3	1	5	0	1	0
14	14	0	842	134	0	0.0	0	4	0	0	0
15	15	0	844	132	10	0.0	0	3	0	0	0
16	16	0	845	131	20	0.9	2	1	0	1	0
17	17	0	849	127	50	0.4	0	1	0	1	0
18	18	0	851	125	50	0.9	0	1	0	0	0
19	19	0	854	122	0	0.8	1	1	0	1	0
20	20	0	855	121	30	0.9	0	1	0	1	0
21	21	0	867	109	0	0.2	0	3	0	1	0
22	22	0	870	106	10	0.0	0	3	0	0	0
23	23	0	872	104	0	0.2	0	3	0	0	0
24	24	0	876	100	10	0.3	0	3	0	0	0
25	25	0	878	98	30	0.6	1	1	1	1	0
26	26	0	882	94	50	0.8	0	1	0	0	0
27	27	0	886	90	30	0.3	0	1	0	1	0
28	28	0	889	87	20	0.5	0	1	0	1	0
29	29	0	892	84	30	0.6	0	1	0	1	0
30	30	0	896	80	10	1.0	0	3	0	1	0

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> # ========================================
> # ASSESSMENT 8: K-MEANS CLUSTERING
> # ========================================
>
> # 1. Load the dataset
> data <- read.csv("assessment.csv")
>
> # 2. View the dataset
> View(data)
> head(data)
  rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1        1      0  786     190    10 0.1       0     3    0     0      0
2        2      0  794     182    10 0.0       0     3    0     0      0
3        3      0  798     178    20 0.0       0     1    0     0      0
4        4      0  800     176    70 0.3       0     1    0     1      0
5        5      0  804     172    40 0.3       0     1    0     1      0
6        6      0  808     168    60 0.3       0     1    0     1      0
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
>
> # 3. Check missing values
> colSums(is.na(data))
rownames   Gunnel     Time  Fromlow    Slope       Rw  Amphiso    Subst     Pool    Water   Cobble
       0        0        0        0        0        0        0        0        0        0        0
>
> # 4. Check dependent variable distribution
> # Replace 'Target' with your dependent variable name
> table(data$Target)
< table of extent 0 >
>
> # If you don't know the dependent variable, view column names
> names(data)
 [1] "rownames" "Gunnel"   "Time"     "Fromlow"  "Slope"    "Rw"       "Amphiso"  "Subst"    "Pool"     "Water"
[11] "Cobble"
```

```
> # If you don't know the dependent variable, view column names
> names(data)
 [1] "rownames" "Gunnel"   "Time"     "Fromlow"  "Slope"    "Rw"       "Amphiso"  "Subst"    "Pool"     "Water"
[11] "Cobble"
>
> # ==========================================
> # 5. Select only numerical features
> # ==========================================
>
> num_data <- data[, sapply(data, is.numeric)]
>
> # View numerical data
> View(num_data)
>
> # ==========================================
> # 6. Standardize the data
> # ==========================================
>
> scaled_data <- scale(num_data)
>
> # View standardized data
> head(scaled_data)
     rownames     Gunnel      Time   Fromlow      Slope         Rw    Amphiso       Subst       Pool      Water
[1,] -1.730419 -0.1762627 0.5944359 0.4910469 0.03122942 -1.0137551 -0.5163247 -0.03785446 -0.5033727 -1.0174257
[2,] -1.728244 -0.1762627 0.6533228 0.3942676 0.03122942 -1.2511280 -0.5163247 -0.03785446 -0.5033727 -1.0174257
[3,] -1.726069 -0.1762627 0.6827663 0.3458780 0.66056149 -1.2511280 -0.5163247 -0.75528659 -0.5033727 -1.0174257
[4,] -1.723894 -0.1762627 0.6974880 0.3216832 3.80722182 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
[5,] -1.721718 -0.1762627 0.7269314 0.2732935 1.91922562 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
[6,] -1.719543 -0.1762627 0.7563749 0.2249039 3.17788976 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
       Cobble
[1,] -0.4856713
[2,] -0.4856713
[3,] -0.4856713
[4,] -0.4856713
[5,] -0.4856713
[6,] -0.4856713
>
> # ==========================================
> # 7. Apply K-Means with K = 2
> # ==========================================
>
> set.seed(123)
>
> kmeans_2 <- kmeans(scaled_data, centers = 2, nstart = 25)
>
> # ==========================================
```

```
> # ==========================================
> # 8. Display K-Means results
> # ==========================================
>
> kmeans_2
K-means clustering with 2 clusters of sizes 1288, 304

Cluster means:
     rownames     Gunnel         Time     Fromlow       Slope          Rw    Amphiso      Subst        Pool
1 -0.01501740 -0.1036410 -0.004279985  0.03125144  0.06054613 -0.03661928 -0.1069849 -0.4288772 -0.02224503
2  0.06362636  0.4391107  0.018133620 -0.13240740 -0.25652439  0.15515010  0.4532780  1.8170852  0.09424868
        Water     Cobble
1 -0.04087335 -0.4856713
2  0.17317391  2.0577125

Clustering vector:
   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 2 1 1 1 1 2
  [56] 1 2 2 1 1 1 2 1 1 1 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 1 2
 [111] 1 1 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 2 1 1 1 1 2 1 1 2 1 1 1 2 1 1 2 1
 [166] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1
 [221] 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [276] 2 1 1 2 2 1 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 2 1 1 1 1 1 1 2 1 2 2 1 1 1 1 2 1 1 1 1 2 1 1 2 2 2 2 1 1 1 1 1 2 1
 [331] 2 1 1 1 1 2 1 1 1 1 2 2 1 1 2 2 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2
 [386] 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 1 2 1 1 1 1 1
 [441] 1 1 1 1 1 1 2 1 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 1 1 1 1 1 1 1
 [496] 1 1 1 1 1 2 2 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1
 [551] 1 1 1 1 1 1 2 2 1 1 1 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1
 [606] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [661] 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 2 1 2 1 1 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2
 [716] 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2
 [771] 2 1 2 2 2 1 2 2 2 2 2 2 2 2 1 2 2 1 2 1 2 2 1 1 2 1 1 2 1 2 2 2 1 1 1 1 1 1 1 1 1 2 2 2 2 2 1 2 2 2 1 2 2 1 2
 [826] 1 1 2 1 1 1 2 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1
 [881] 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 2 1 1
 [936] 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [991] 1 1 1 1 1 1 1 1 1 1
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]

Within cluster sum of squares by cluster:
[1] 10979.08  3484.00
 (between_SS / total_SS =  17.4 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss" "betweenss"    "size"
[8] "iter"         "ifault"
```

```
>
> # Cluster assignment for every observation
> kmeans_2$cluster
   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 2 1 1 1 2
  [56] 1 2 2 1 1 1 2 1 1 1 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 1 2 1 2 2
 [111] 1 1 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 2 1 1 1 1 2 1 1 1 2 1 1 2 1 1 2 1 1 1
 [166] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1
 [221] 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [276] 2 1 1 2 2 1 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 2 1 1 1 1 1 1 2 1 2 2 1 1 1 2 1 1 1 1 2 1 1 2 2 2 2 2 1 1 1 1 1 2 1
 [331] 2 1 1 1 1 2 1 1 1 1 2 2 1 1 2 2 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 2 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2
 [386] 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 1 2 1 1 1 1 1
 [441] 1 1 1 1 1 1 2 1 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 1 1 1 1 1 1 1 1
 [496] 1 1 1 1 1 2 2 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [551] 1 1 1 1 1 1 2 2 1 1 1 1 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1
 [606] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [661] 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 2 1 2 1 1 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2 2
 [716] 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2
 [771] 2 1 2 2 2 1 2 2 2 2 2 2 2 1 2 2 1 2 1 2 2 2 1 1 2 1 1 2 1 2 2 2 1 1 1 1 1 1 1 1 1 2 2 2 2 1 2 2 2 1 2 2 1 2
 [826] 1 1 2 1 1 1 2 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1
 [881] 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 2 1 1
 [936] 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [991] 1 1 1 1 1 1 1 1 1 1
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]
>
> # ============================================
> # 9. Display cluster centers
> # ============================================
>
> kmeans_2$centers
     rownames     Gunnel         Time     Fromlow       Slope          Rw    Amphiso      Subst        Pool
1 -0.01501740 -0.1036410 -0.004279985  0.03125144  0.06054613 -0.03661928 -0.1069849 -0.4288772 -0.02224503
2  0.06362636  0.4391107  0.018133620 -0.13240740 -0.25652439  0.15515010  0.4532780  1.8170852  0.09424868
       Water     Cobble
1 -0.04087335 -0.4856713
2  0.17317391  2.0577125
>
> # ============================================
> # 10. Calculate WCSS for K = 1 to 10
> # ============================================
>
> wcss <- numeric(10)
>
> for (k in 1:10) {
+
+   set.seed(123)
+
+
+   km <- kmeans(scaled_data,
+                centers = k,
+                nstart = 25)
+
+   wcss[k] <- km$tot.withinss
+ }
>
> # Display WCSS values
> wcss
 [1] 17501.000 14463.080 12566.490 11067.934  9930.526  8958.508  8284.558  7678.426  7268.908  6886.292
>
> # ============================================
> # 11. Display Elbow Plot
> # ============================================
>
> plot(1:10,
+      wcss,
+      type = "b",
+      pch = 19,
+      xlab = "Number of Clusters (K)",
+      ylab = "WCSS",
+      main = "Elbow Method for K-Means")
>
> # ============================================
> # 12. Apply K-Means based on elbow
> # ============================================
>
> # Change this value according to the elbow plot
> optimal_k <- 3
>
> set.seed(123)
>
> final_kmeans <- kmeans(scaled_data,
+                        centers = optimal_k,
+                        nstart = 25)
>
> # Display final result
> final_kmeans
K-means clustering with 3 clusters of sizes 304, 713, 575

Cluster means:
     rownames      Gunnel        Time    Fromlow        Slope         Rw    Amphiso      Subst        Pool
1  0.06362636  0.43911070  0.01813362 -0.1324074 -0.256524389  0.1551501  0.4532780  1.8170852  0.09424868
2 -0.11748083 -0.14346584 -0.08332096  0.2216632  0.111550902 -0.4537815 -0.3572020 -0.3130546 -0.50337267
3  0.11203725 -0.05425827  0.09373082 -0.2048592 -0.002699789  0.4806619  0.2032843 -0.5724974  0.57435325
```

```
       Water     Cobble
1  0.1731739  2.0577125
2 -0.8463450 -0.4856713
3  0.9579115 -0.4856713

Clustering vector:
   [1] 2 2 2 2 2 2 2 3 3 2 3 3 3 2 2 3 3 2 3 3 3 2 2 2 3 2 3 3 3 3 3 2 2 3 3 3 2 3 3 2 2 2 2 1 1 1 2 2 1 3 3 3 3 1
  [56] 3 1 1 3 3 3 2 1 3 2 3 3 2 1 3 3 2 2 1 3 3 3 3 3 3 3 3 2 2 2 2 2 2 2 2 2 3 3 3 2 3 3 3 3 1 2 3 3 2 3 3 2 2 2 1 2 1 3 1 1
 [111] 2 2 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 3 3 3 3 2 3 3 2 2 2 2 2 2 1 1 1 3 2 1 2 2 2 2 1 3 3 1 3 3 3 1 2 3 1 2 1 2 2
 [166] 2 2 2 3 3 2 2 2 2 2 3 3 3 3 3 1 1 1 2 2 2 3 3 2 2 1 1 1 2 2 2 3 2 2 2 2 2 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2
 [221] 2 2 2 2 2 2 2 2 2 2 2 2 1 3 3 2 2 2 1 2 1 2 2 2 2 2 2 3 2 2 2 1 2 2 1 2 2 3 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [276] 1 2 2 1 1 2 3 1 1 2 2 1 2 1 2 2 1 1 2 2 2 2 1 2 2 2 2 2 2 2 1 2 1 1 2 2 3 2 1 2 2 2 2 1 2 2 1 1 1 1 3 2 2 3 3 1 3
 [331] 1 2 3 3 3 1 2 3 2 2 1 1 3 3 1 1 3 2 1 1 1 2 2 3 3 3 2 2 3 3 2 2 3 2 2 2 1 1 2 1 2 2 3 2 3 2 3 3 2 3 2 2 3 3 3 2 2 2 1 1
 [386] 1 1 1 1 2 3 2 3 2 3 2 3 3 3 3 3 2 2 2 2 2 2 2 1 3 3 3 2 2 3 2 2 3 2 2 2 3 3 3 3 3 2 3 3 2 3 3 2 2 3 2 2 2 3 2 2 2
 [441] 3 3 3 3 3 3 3 1 3 1 1 1 1 1 1 3 3 3 3 3 3 3 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 2 3 2 3 2 2 2 3 3 3 3 1 1 2 3 3 3 3 3 2 2
 [496] 3 3 3 3 3 2 1 1 3 3 1 3 3 2 2 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 3 3 3 2 2 1 3 1 2 3 3 1 3 1 2 3 2 2 2 3 2 2 2 2 2 2 2
 [551] 2 2 2 2 2 3 1 1 3 2 3 2 3 1 3 2 3 2 2 3 1 3 2 3 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 2 2 2 3 2 2 2 2 1 1 2
 [606] 2 3 2 3 2 2 2 2 3 3 2 2 2 2 3 3 3 3 3 2 2 3 2 2 3 2 2 2 3 3 3 3 3 2 3 3 2 3 2 2 3 2 2 3 2 3 2 2 2 3 3 3 2 2 2
 [661] 3 2 2 2 2 3 3 2 2 2 3 3 1 2 3 3 3 3 2 3 1 1 2 1 3 3 1 1 1 1 1 3 3 3 3 2 2 2 3 3 3 3 2 1 2 2 3 3 3 3 2 2 3 3 1 1 1
 [716] 1 1 3 3 3 1 3 3 2 3 3 3 3 3 3 3 3 3 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 1 2 3 3 3 3 1 3 3 2 3 3 3 3 3 3 3 3 1
 [771] 1 3 1 1 1 3 1 1 1 1 1 1 1 3 1 1 3 1 2 1 1 3 3 1 3 3 1 3 1 1 1 3 3 3 3 3 2 3 3 3 2 1 1 1 1 3 1 1 1 2 1 1 2 1 2 1
 [826] 3 2 1 3 3 3 1 2 2 2 2 2 1 1 1 3 3 2 3 3 3 3 1 2 2 2 3 3 3 3 3 3 3 3 3 3 3 1 3 2 3 2 3 3 3 3 3 3 3 3 3 1 1 2 3
 [881] 3 3 1 3 3 1 3 2 2 2 2 3 3 3 3 1 3 2 2 3 3 2 3 2 3 1 1 1 3 3 2 3 3 2 2 2 2 3 2 3 2 2 3 2 2 2 2 2 1 2 2 2 1 3 2
 [936] 2 2 2 2 2 2 1 2 2 1 2 2 2 3 3 2 3 1 3 1 2 3 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [991] 2 2 2 2 2 2 2 2 2 2
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]

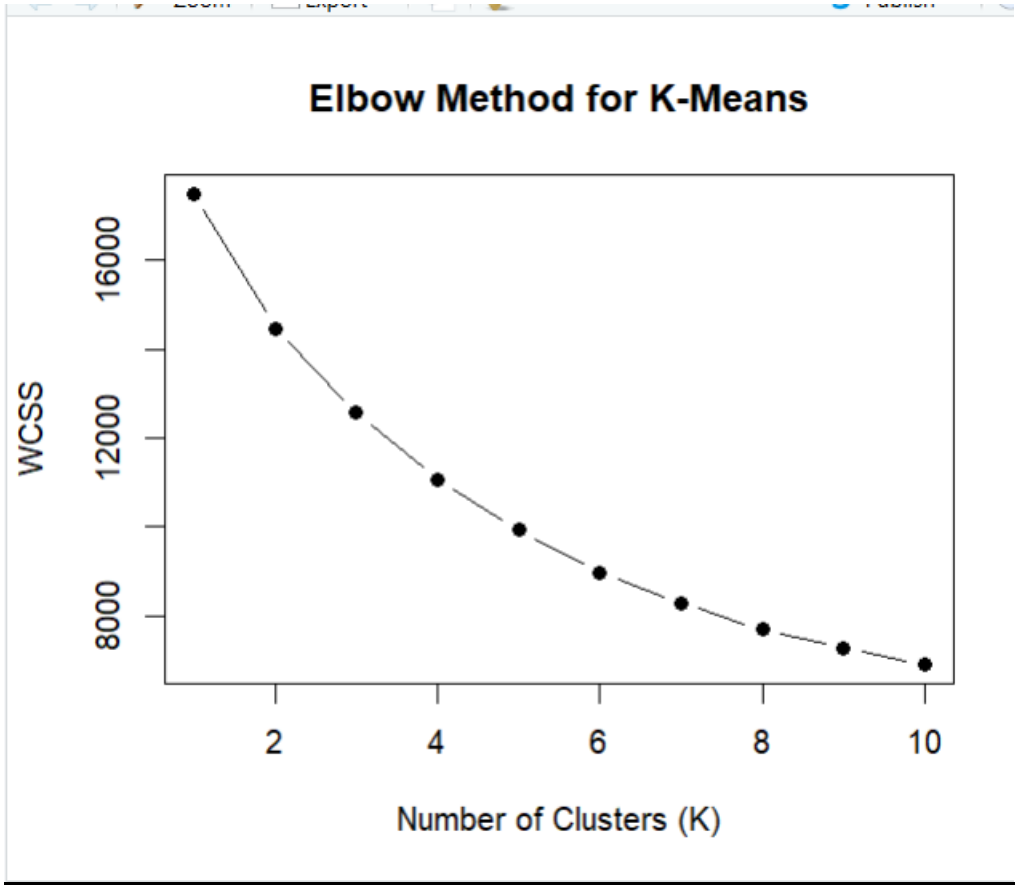
Within cluster sum of squares by cluster:
[1] 3484.000 4469.027 4613.463
 (between_SS / total_SS =  28.2 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss" "betweenss"    "size"
[8] "iter"         "ifault"
>
```

```
>
> # Display final cluster assignments
> final_kmeans$cluster
   [1] 2 2 2 2 2 2 2 3 3 2 3 3 3 2 2 3 3 2 3 3 3 2 2 2 3 2 3 3 3 3 3 2 2 3 3 3 2 3 3 2 2 2 2 1 1 1 2 2 1 3 3 3 3 1
  [56] 3 1 1 3 3 3 2 1 3 2 3 3 2 1 3 3 2 2 1 3 3 3 3 3 3 3 3 2 2 2 2 2 2 2 2 2 3 3 3 2 3 3 3 3 1 2 3 3 2 3 3 2 2 2 1 2 1 3 1 1
 [111] 2 2 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 3 3 3 3 2 3 3 2 2 2 2 2 2 1 1 1 3 2 1 2 2 2 2 1 3 3 1 3 3 3 1 2 3 1 2 1 2 2
 [166] 2 2 2 3 3 2 2 2 2 2 3 3 3 3 3 1 1 1 2 2 2 3 3 2 2 1 1 1 2 2 2 3 2 2 2 2 2 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2
 [221] 2 2 2 2 2 2 2 2 2 2 2 2 1 3 3 2 2 2 1 2 1 2 2 2 2 2 2 3 2 2 2 1 2 2 1 2 2 3 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [276] 1 2 2 1 1 2 3 1 1 2 2 1 2 1 2 2 1 1 2 2 2 2 1 2 2 2 2 2 2 2 1 2 1 1 2 2 3 2 1 2 2 2 2 1 2 2 1 1 1 1 3 2 2 3 3 1 3
 [331] 1 2 3 3 3 1 2 3 2 2 1 1 3 3 1 1 3 2 1 1 1 2 2 3 3 3 2 2 3 3 2 2 3 2 2 2 1 1 2 1 2 2 3 2 3 2 3 3 2 3 2 2 3 3 3 2 2 2 1 1
 [386] 1 1 1 1 2 3 2 3 2 3 2 3 3 3 3 3 2 2 2 2 2 2 2 1 3 3 3 2 2 3 2 2 3 2 2 2 3 3 3 3 3 2 3 3 2 3 3 2 2 3 2 2 2 3 2 2 2
 [441] 3 3 3 3 3 3 3 1 3 1 1 1 1 1 1 3 3 3 3 3 3 3 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 2 3 2 3 2 2 2 3 3 3 3 1 1 2 3 3 3 3 3 2 2
 [496] 3 3 3 3 3 2 1 1 3 3 1 3 3 2 2 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 3 3 3 2 2 1 3 1 2 3 3 1 3 1 2 3 2 2 2 3 2 2 2 2 2 2 2
 [551] 2 2 2 2 2 3 1 1 3 2 3 2 3 1 3 2 3 2 2 3 1 3 2 3 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 2 2 2 3 2 2 2 2 1 1 2
 [606] 2 3 2 3 2 2 2 2 3 3 2 2 2 2 3 3 3 3 3 2 2 3 2 2 3 2 2 2 3 3 3 3 3 2 3 3 2 3 2 2 3 2 2 3 2 3 2 2 2 3 3 3 2 2 2
 [661] 3 2 2 2 2 3 3 2 2 2 3 3 1 2 3 3 3 3 2 3 1 1 2 1 3 3 1 1 1 1 1 3 3 3 3 2 2 2 3 3 3 3 2 1 2 2 3 3 3 3 2 2 3 3 1 1 1
 [716] 1 1 3 3 3 1 3 3 2 3 3 3 3 3 3 3 3 3 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 1 2 3 3 3 3 1 3 3 2 3 3 3 3 3 3 3 3 1
 [771] 1 3 1 1 1 3 1 1 1 1 1 1 1 3 1 1 3 1 2 1 1 3 3 1 3 3 1 3 1 1 1 3 3 3 3 3 2 3 3 3 2 1 1 1 1 3 1 1 1 2 1 1 2 1 2 1
 [826] 3 2 1 3 3 3 1 2 2 2 2 2 1 1 1 3 3 2 3 3 3 3 1 2 2 2 3 3 3 3 3 3 3 3 3 3 3 1 3 2 3 2 3 3 3 3 3 3 3 3 3 1 1 2 3
 [881] 3 3 1 3 3 1 3 2 2 2 2 3 3 3 3 1 3 2 2 3 3 2 3 2 3 1 1 1 3 3 2 3 3 2 2 2 2 3 2 3 2 2 3 2 2 2 2 2 1 2 2 2 1 3 2
 [936] 2 2 2 2 2 2 1 2 2 1 2 2 2 3 3 2 3 1 3 1 2 3 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [991] 2 2 2 2 2 2 2 2 2 2
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]
>
> # Display final cluster centers
> final_kmeans$centers
     rownames      Gunnel        Time    Fromlow        Slope         Rw    Amphiso      Subst        Pool
1  0.06362636  0.43911070  0.01813362 -0.1324074 -0.256524389  0.1551501  0.4532780  1.8170852  0.09424868
2 -0.11748083 -0.14346584 -0.08332096  0.2216632  0.111550902 -0.4537815 -0.3572020 -0.3130546 -0.50337267
3  0.11203725 -0.05425827  0.09373082 -0.2048592 -0.002699789  0.4806619  0.2032843 -0.5724974  0.57435325
       Water     Cobble
1  0.1731739  2.0577125
2 -0.8463450 -0.4856713
3  0.9579115 -0.4856713
>
> # Number of observations in each cluster
> table(final_kmeans$cluster)

  1   2   3
304 713 575
>
```


Elbow Method for K-Means
WCSS
16000
12000
8000
2
4
6
8
10
Number of Clusters (K)

Part B — Hierarchical Clustering

CODE:

assessment_8.R*

Source on Save

```r
#23BDS0334
#ArjunMishra
# ============================================
# HIERARCHICAL CLUSTERING
# ============================================

# 1. Load the dataset
data <- read.csv("assessment.csv")

# 2. View the dataset
View(data)
head(data)
str(data)

# ============================================
# 3. Check missing values
# ============================================

colSums(is.na(data))

# ============================================
# 4. Select only numerical attributes
# ============================================

num_data <- data[, sapply(data, is.numeric)]

# View numerical attributes
View(num_data)

# ============================================
# 5. Standardize the data
# ============================================

scaled_data <- scale(num_data)

# Display standardized data
head(scaled_data)

# ============================================
# 6. Calculate Euclidean distance
# ============================================

distance_matrix <- dist(scaled_data,
                        method = "euclidean")

```

```r
# ============================================
# 6. Calculate Euclidean distance
# ============================================

distance_matrix <- dist(scaled_data,
                        method = "euclidean")

# Display distance matrix
distance_matrix

# ============================================
# 7. Hierarchical Clustering using Ward's method
# ============================================

hc <- hclust(distance_matrix,
             method = "ward.D2")

# Display clustering result
hc

# ============================================
# 8. Plot the dendrogram
# ============================================

plot(hc,
     main = "Hierarchical Clustering Dendrogram",
     xlab = "Observations",
     ylab = "Height",
     sub = "",
     cex = 0.6)

# ============================================
# 9. Divide data into 3 clusters
# ============================================

clusters <- cutree(hc, k = 3)

# Display cluster assignments
clusters

# ============================================
# 10. Display number of samples in each cluster
# ============================================

table(clusters)
```

13:10 (Untitled)

OUTPUT:

```
> #23BDS0334
> #ArjunMishra
> # ============================================
> # HIERARCHICAL CLUSTERING
> # ============================================
>
> # 1. Load the dataset
> data <- read.csv("assessment.csv")
>
> # 2. View the dataset
> View(data)
> head(data)
  rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1        1      0  786     190    10 0.1       0     3    0     0      0
2        2      0  794     182    10 0.0       0     3    0     0      0
3        3      0  798     178    20 0.0       0     1    0     0      0
4        4      0  800     176    70 0.3       0     1    0     1      0
5        5      0  804     172    40 0.3       0     1    0     1      0
6        6      0  808     168    60 0.3       0     1    0     1      0
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
>
> # ============================================
> # 3. Check missing values
> # ============================================
>
> colSums(is.na(data))
rownames   Gunnel     Time  Fromlow    Slope       Rw  Amphiso    Subst     Pool    Water   Cobble
       0        0        0        0        0        0        0        0        0        0        0
>
> # ============================================
> # 4. Select only numerical attributes
> # ============================================
>
```

```
>
> num_data <- data[, sapply(data, is.numeric)]
>
> # View numerical attributes
> View(num_data)
>
> # ============================================
> # 5. Standardize the data
> # ============================================
>
> scaled_data <- scale(num_data)
>
> # Display standardized data
> head(scaled_data)
     rownames     Gunnel       Time   Fromlow      Slope         Rw    Amphiso       Subst       Pool      Water
[1,] -1.730419 -0.1762627 0.5944359 0.4910469 0.03122942 -1.0137551 -0.5163247 -0.03785446 -0.5033727 -1.0174257
[2,] -1.728244 -0.1762627 0.6533228 0.3942676 0.03122942 -1.2511280 -0.5163247 -0.03785446 -0.5033727 -1.0174257
[3,] -1.726069 -0.1762627 0.6827663 0.3458780 0.66056149 -1.2511280 -0.5163247 -0.75528659 -0.5033727 -1.0174257
[4,] -1.723894 -0.1762627 0.6974880 0.3216832 3.80722182 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
[5,] -1.721718 -0.1762627 0.7269314 0.2732935 1.91922562 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
[6,] -1.719543 -0.1762627 0.7563749 0.2249039 3.17788976 -0.5390094 -0.5163247 -0.75528659 -0.5033727  0.9822554
       Cobble
[1,] -0.4856713
[2,] -0.4856713
[3,] -0.4856713
[4,] -0.4856713
[5,] -0.4856713
[6,] -0.4856713
>
> # ============================================
> # 6. Calculate Euclidean distance
> # ============================================
>
> distance_matrix <- dist(scaled_data,
+                         method = "euclidean")
>
> # Display distance matrix
> distance_matrix
                1           2           3           4           5           6           7           8
2     0.263029475
                9          10          11          12          13          14          15          16
2
               17          18          19          20          21          22          23          24
2
               25          26          27          28          29          30          31          32
2
```

```
> distance_matrix
          1         2         3         4         5         6         7         8
2 0.263029475
          9        10        11        12        13        14        15        16
2
         17        18        19        20        21        22        23        24
2
         25        26        27        28        29        30        31        32
2
         33        34        35        36        37        38        39        40
2
         41        42        43        44        45        46        47        48
2
         49        50        51        52        53        54        55        56
2
         57        58        59        60        61        62        63        64
2
         65        66        67        68        69        70        71        72
2
         73        74        75        76        77        78        79        80
2
         81        82        83        84        85        86        87        88
2
         89        90        91        92        93        94        95        96
2
         97        98        99       100       101       102       103       104
2
        105       106       107       108       109       110       111       112
2
        113       114       115       116       117       118       119       120
2
        121       122       123       124       125       126       127       128
2
        129       130       131       132       133       134       135       136
2
        137       138       139       140       141       142       143       144
2
        145       146       147       148       149       150       151       152
2
        153       154       155       156       157       158       159       160
2
        161       162       163       164       165       166       167       168
2
        169       170       171       172       173       174       175       176
2
        177       178       179       180       181       182       183       184
```

```
2
        177       178       179       180       181       182       183       184
2
        185       186       187       188       189       190       191       192
2
        193       194       195       196       197       198       199       200
2
        201       202       203       204       205       206       207       208
2
        209       210       211       212       213       214       215       216
2
        217       218       219       220       221       222       223       224
2
        225       226       227       228       229       230       231       232
2
        233       234       235       236       237       238       239       240
2
        241       242       243       244       245       246       247       248
2
        249       250       251       252       253       254       255       256
2
        257       258       259       260       261       262       263       264
2
        265       266       267       268       269       270       271       272
2
        273       274       275       276       277       278       279       280
2
        281       282       283       284       285       286       287       288
2
        289       290       291       292       293       294       295       296
2
        297       298       299       300       301       302       303       304
2
        305       306       307       308       309       310       311       312
2
        313       314       315       316       317       318       319       320
2
        321       322       323       324       325       326       327       328
2
        329       330       331       332       333       334       335       336
2
        337       338       339       340       341       342       343       344
2
        345       346       347       348       349       350       351       352
2
        353       354       355       356       357       358       359       360
2
```

2	361	362	363	364	365	366	367	368
2	369	370	371	372	373	374	375	376
2	377	378	379	380	381	382	383	384
2	385	386	387	388	389	390	391	392
2	393	394	395	396	397	398	399	400
2	401	402	403	404	405	406	407	408
2	409	410	411	412	413	414	415	416
2	417	418	419	420	421	422	423	424
2	425	426	427	428	429	430	431	432
2	433	434	435	436	437	438	439	440
2	441	442	443	444	445	446	447	448
2	449	450	451	452	453	454	455	456
2	457	458	459	460	461	462	463	464
2	465	466	467	468	469	470	471	472
2	473	474	475	476	477	478	479	480
2	481	482	483	484	485	486	487	488
2	489	490	491	492	493	494	495	496
2	497	498	499	500	501	502	503	504
2	505	506	507	508	509	510	511	512
2	513	514	515	516	517	518	519	520
2	521	522	523	524	525	526	527	528
2	529	530	531	532	533	534	535	536
2	537	538	539	540	541	542	543	544

2	545	546	547	548	549	550	551	552
2	553	554	555	556	557	558	559	560
2	561	562	563	564	565	566	567	568
2	569	570	571	572	573	574	575	576
2	577	578	579	580	581	582	583	584
2	585	586	587	588	589	590	591	592
2	593	594	595	596	597	598	599	600
2	601	602	603	604	605	606	607	608
2	609	610	611	612	613	614	615	616
2	617	618	619	620	621	622	623	624
2	625	626	627	628	629	630	631	632
2	633	634	635	636	637	638	639	640
2	641	642	643	644	645	646	647	648
2	649	650	651	652	653	654	655	656
2	657	658	659	660	661	662	663	664
2	665	666	667	668	669	670	671	672
2	673	674	675	676	677	678	679	680
2	681	682	683	684	685	686	687	688
2	689	690	691	692	693	694	695	696
2	697	698	699	700	701	702	703	704
2	705	706	707	708	709	710	711	712
2	713	714	715	716	717	718	719	720
2	721	722	723	724	725	726	727	728

```
          721  722  723  724  725  726  727  728
2
          729  730  731  732  733  734  735  736
2
          737  738  739  740  741  742  743  744
2
          745  746  747  748  749  750  751  752
2
          753  754  755  756  757  758  759  760
2
          761  762  763  764  765  766  767  768
2
          769  770  771  772  773  774  775  776
2
          777  778  779  780  781  782  783  784
2
          785  786  787  788  789  790  791  792
2
          793  794  795  796  797  798  799  800
2
          801  802  803  804  805  806  807  808
2
          809  810  811  812  813  814  815  816
2
          817  818  819  820  821  822  823  824
2
          825  826  827  828  829  830  831  832
2
          833  834  835  836  837  838  839  840
2
          841  842  843  844  845  846  847  848
2
          849  850  851  852  853  854  855  856
2
          857  858  859  860  861  862  863  864
2
          865  866  867  868  869  870  871  872
2
          873  874  875  876  877  878  879  880
2
          881  882  883  884  885  886  887  888
2
          889  890  891  892  893  894  895  896
2
          897  898  899  900  901  902  903  904
```

```
2
          897  898  899  900  901  902  903  904
2
          905  906  907  908  909  910  911  912
2
          913  914  915  916  917  918  919  920
2
          921  922  923  924  925  926  927  928
2
          929  930  931  932  933  934  935  936
2
          937  938  939  940  941  942  943  944
2
          945  946  947  948  949  950  951  952
2
          953  954  955  956  957  958  959  960
2
          961  962  963  964  965  966  967  968
2
          969  970  971  972  973  974  975  976
2
          977  978  979  980  981  982  983  984
2
          985  986  987  988  989  990  991  992
2
          993  994  995  996  997  998  999 1000
2
 [ reached 'max' / getOption("max.print") -- omitted 1590 rows and 591 columns ]
>
> # ============================================
> # 7. Hierarchical Clustering using Ward's method
> # ============================================
>
> hc <- hclust(distance_matrix,
+              method = "ward.D2")
>
> # Display clustering result
> hc

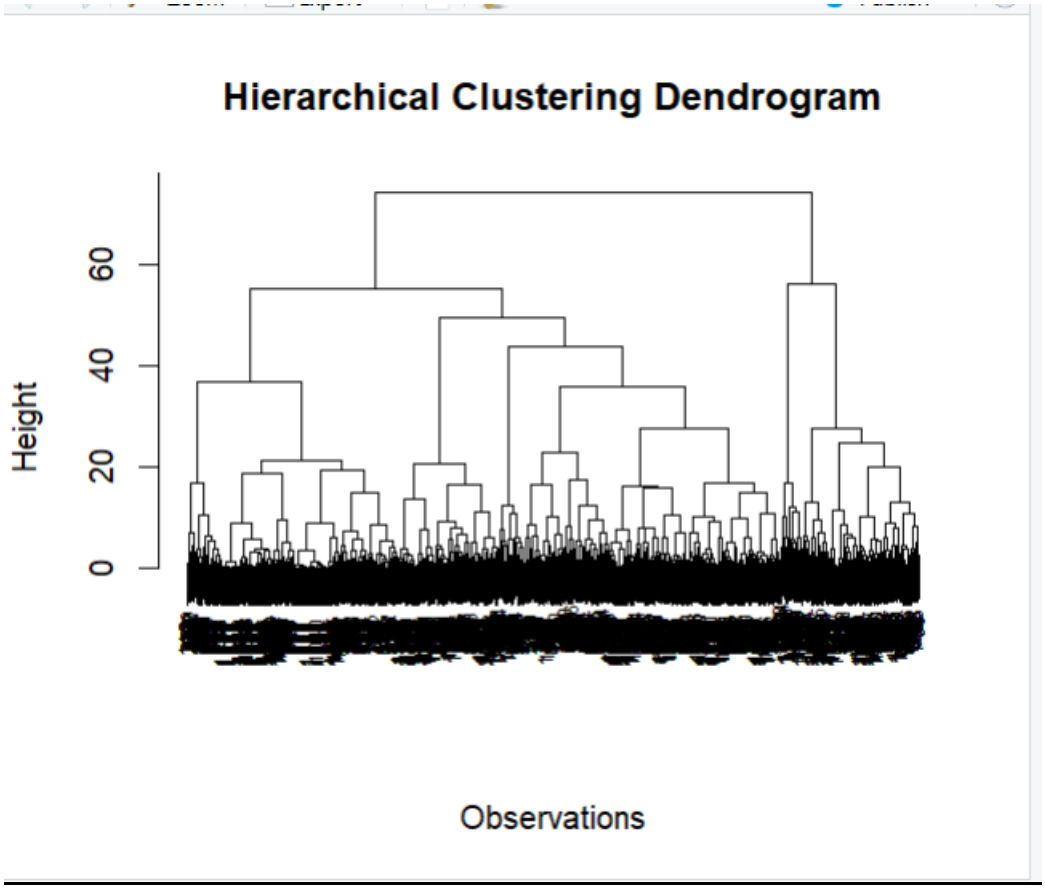
Call:
hclust(d = distance_matrix, method = "ward.D2")

Cluster method   : ward.D2
Distance         : euclidean
Number of objects: 1592
```

```
Number of objects: 1992

>
> # ============================================
> # 8. Plot the dendrogram
> # ============================================
>
> plot(hc,
+      main = "Hierarchical Clustering Dendrogram",
+      xlab = "Observations",
+      ylab = "Height",
+      sub = "",
+      cex = 0.6)
>
> # ============================================
> # 9. Divide data into 3 clusters
> # ============================================
>
> clusters <- cutree(hc, k = 3)
>
> # Display cluster assignments
> clusters
   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 2 1 1 1 1 2
  [56] 1 2 2 1 1 1 2 1 1 1 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 3 1 3 3 3 1
 [111] 1 1 3 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 1 2 1 1 1 1 2 1 1 2 1 1 1 1 2 1 1 2 1 2 1 1
 [166] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [221] 1 1 1 1 3 1 1 1 1 1 1 1 3 3 3 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [276] 2 1 1 2 2 1 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 2 1 1 1 1 1 1 1 2 1 2 2 2 1 1 1 1 2 1 1 1 1 2 1 1 1 2 1 1 2 1 3 2 1 1 1 1 1 1 1 1
 [331] 3 1 1 1 1 3 1 1 1 3 2 2 1 1 2 2 1 1 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2
 [386] 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 1 2 1 1 1 1 1 1 1
 [441] 1 1 1 1 1 1 2 1 2 3 2 2 3 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 1 1 1 1 1 1 1 1 1 1
 [496] 1 1 1 1 1 2 2 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 3 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1
 [551] 1 1 1 1 1 1 2 2 1 1 1 1 1 2 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1
 [606] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [661] 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 2 1 2 1 1 3 2 2 2 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 3 1 1 1 1 1 1 2 2 2
 [716] 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 2
 [771] 2 1 2 2 2 1 2 2 2 2 2 2 2 1 2 2 1 2 1 2 2 1 1 2 1 1 2 1 2 2 2 1 1 1 1 1 1 1 1 1 2 2 2 2 1 2 2 2 1 2 2 1 2
 [826] 1 1 2 1 1 1 1 1 1 1 1 1 1 2 1 3 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 1 1
 [881] 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 2 1 1
 [936] 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [991] 1 1 1 1 1 1 1 1 1 1
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]
>
> # ============================================
> # 10. Display number of samples in each cluster
> # ============================================
```

```
 [991] 1 1 1 1 1 1 1 1 1 1
 [ reached 'max' / getOption("max.print") -- omitted 592 entries ]
>
> # ============================================
> # 10. Display number of samples in each cluster
> # ============================================
>
> table(clusters)
clusters
   1    2    3
1291  253   48
>
```


Hierarchical Clustering Dendrogram
Height
0
20
40
60
Observations